



**Department of Computer Science and Engineering,
PES University, Bangalore, India**

**Lecture Notes
Problem Solving With C
UE24CS151B**

***Lecture #8
Environment Variables***

**By,
Prof. Sindhu R Pai,
Theory Anchor, Feb-May, 2025
Assistant Professor
Dept. of CSE, PESU**

**Many Thanks to
Dr. Shylaja S S (Director, CCBD and CDSAML Research Center, PES University)
Prof. Nitin V Pujari (Dean, Internal Quality Assurance Cell, PES University)**

Unit #: 3**Unit Name: File Handling and Portable Programming****Topic: Environment Variables**

Course objectives: The objective(s) of this course is to make students

- Acquire knowledge on how to solve relevant and logical problems using computing Machine.
- Map algorithmic solutions to relevant features of C programming language constructs.
- Gain knowledge about C constructs and its associated ecosystem.
- Appreciate and gain knowledge about the issues with C Standards and it's respective behaviours.

Course outcomes: At the end of the course, the student will be able to:

- Understand and Apply algorithmic solutions to counting problems using appropriate C Constructs.
- Understand, Analyze and Apply sorting and Searching techniques.
- Understand, Analyze and Apply text processing and string manipulation methods using Arrays, Pointers and functions.
- Understand user defined type creation and implement the same using C structures, unions and other ways by reading and storing the data in secondary systems which are portable.

Sindhu R Pai

Theory Anchor, Feb - May, 2025

Dept. of CSE,

PES University

Introduction

Environment variables are dynamic values that affect the way processes run on a computer. They are part of the environment in which a process executes and can influence the behavior of software and operating systems. They are typically used for **storing system-wide or user-specific settings**, pass configuration data to applications (like file paths, user details, or system preferences) and **control program behavior without hard coding** values in the program.

Examples of common environment variables include:

PATH – Specifies directories to search for executable files.

HOME or USERPROFILE – Indicates the user's home directory.

TEMP or TMP – Path for storing temporary files.

Widely used in operating systems like **Windows, Linux, and macOS**. Can be accessed or modified using shell commands or using C programs.

Standard Environment Variables

Environment variables store information such as the **user's home directory, terminal type, and current locale**. The collection of all environment variables with assigned values is referred to as the **environment**. These variable names are **case-sensitive and must not include the '=' character**. Their values can be any string, but must not contain NULL character.

The **number of environment variables in a system is not fixed** — it varies depending on **Operating system (Windows, Linux, macOS), User configuration (user-defined variables), System services, installed applications (they may add their own variables) and Shell session or login context**.

There are two ways in which we can display all environment variables.

Method – 1: Using an array of character pointers specified in main()

```
int main(int argc, char* argv[], char* envp[])
```

Method – 2: Using environ variable declared in stdlib.h.

char environ** is represented as an array of strings. Each string is of the format **'name=value'**.

Coding Example_1: Printing two environment variables

```
#include <stdio.h>
int main(int argc, char *argv[], char *envp[]){
    printf("%s\n", envp[0]);
    printf("%s\n", envp[1]);
    return 0;
}
```

```
C:\Users\De11\P2\Unit4>a
ALLUSERSPROFILE=C:\ProgramData
APPDATA=C:\Users\De11\AppData\Roaming
```

Coding Example_2: Printing all environment variables using method-1

```
int main(int argc, char *argv[], char *envp[]){
    int i = 0;
    while(envp[i] != NULL){
        printf("%d %s\n", i, envp[i]); i++;
    }
    return 0;
}
```

Sample only. Many more there.

```
C:\Users\De11\P2\Unit4>a
0 ALLUSERSPROFILE=C:\ProgramData
1 APPDATA=C:\Users\De11\AppData\Roaming
2 CommonProgramFiles=C:\Program Files (x86)\Common Files
3 CommonProgramFiles(x86)=C:\Program Files (x86)\Common Files
4 CommonProgramW6432=C:\Program Files\Common Files
5 COMPUTERNAME=DESKTOP-S8KH1MP
6 ComSpec=C:\Windows\system32\cmd.exe
```

Coding Example_3: Printing all environment variables using method-2

```
#include<stdlib.h>
int main(){
    char **p = environ;   int i = 0;
    while((*p) != NULL) {
        printf("%d %s\n", i, *p); i++;
        p++;
        //sleep(2);
    }
    return 0;
}
```

Sample only. Many more there.

```
C:\Users\De11\P2\Unit4>a
0 ALLUSERSPROFILE=C:\ProgramData
1 APPDATA=C:\Users\De11\AppData\Roaming
2 CommonProgramFiles=C:\Program Files (x86)\Common Files
3 CommonProgramFiles(x86)=C:\Program Files (x86)\Common Files
4 CommonProgramW6432=C:\Program Files\Common Files
5 COMPUTERNAME=DESKTOP-S8KH1MP
6 ComSpec=C:\Windows\system32\cmd.exe
```

Accessing and Modifying Environment Variables

The process of accessing and modifying environment variables is carried out using functions such as `getenv()`, `putenv()` and `setenv()`. Let us discuss these functions in detail.

getenv():

- Used to retrieve the value of a specific environment variable.
- **Prototype:** `char *getenv(const char *name);`
- Returns a pointer to the value associated with name, or NULL if it doesn't exist.

Coding Example_4:

```
int main(){
    char *path = getenv("PATH");
    if (path != NULL)
        printf("PATH: %s\n", path);
    else
        printf("PATH not found\n");
}
```

```
C:\Users\De11>a
PATH: C:\Program Files\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files\Git\cmd;C:\Users\De11\U\Unit4;C:\Users\De11\AppData\Local\Programs\Python\Python311\Scripts\;C:\Users\De11\AppData\Local\Programs\Python\Python311\;C:\Users\De11\AppData\Local\Microsoft\WindowsApps;C:\MinGW\bin;C:\Users\De11\U\Unit4\Desktop;C:\Program Files\JetBrains\PyCharm 2024.2.1\bin;
```

setenv()

- Used to set or update an environment variable.
- **Prototype:** `int setenv(const char *name, const char *value, int overwrite);`
- If overwrite is non-zero, the variable is updated if it exists. If zero, it won't overwrite existing values.
- A null pointer for the value parameter is illegal.
- If the function is successful it returns 0. Otherwise the environment is unchanged and the return value is -1.

Coding Example_5: Appending the PATH with current directory(.)

```
include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main(){
    char *path = getenv("PATH");
    char path1[10000];
    strcpy(path1, path); strcat(path1, ".");
    printf("Before setting %s\n", getenv("PATH"));
    int i = setenv("PATH",path1, 1);
    printf("\n\nAfter setting %s\n", getenv("PATH"));
    printf("status of function is %d", i);
    return 0;
}
```

```
C:\Users\De11>a
Before setting C:\Program Files\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files\Git\cmd;C:\Users\De11\U\Unit4;C:\Users\De11\AppData\Local\Programs\Python\Python311\Scripts\;C:\Users\De11\AppData\Local\Programs\Python\Python311\;C:\Users\De11\AppData\Local\Microsoft\WindowsApps;C:\MinGW\bin;C:\Users\De11\U\Unit4\Desktop;C:\Program Files\JetBrains\PyCharm 2024.2.1\bin;
After setting C:\Program Files\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files\Git\cmd;C:\Users\De11\U\Unit4;C:\Users\De11\AppData\Local\Programs\Python\Python311\Scripts\;C:\Users\De11\AppData\Local\Programs\Python\Python311\;C:\Users\De11\AppData\Local\Microsoft\WindowsApps;C:\MinGW\bin;C:\Users\De11\U\Unit4\Desktop;C:\Program Files\JetBrains\PyCharm 2024.2.1\bin;.
status of function is 0
```

Points to think:

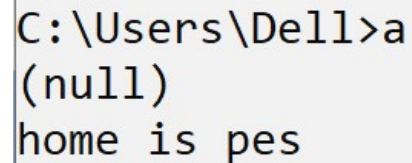
- If the last argument of setenv is 0, will the PATH be updated? - No
- What is the return value of setenv in this case? – 0(success)
- Can you create a new environment variable? If yes, what is the last argument?

putenv()

- Another way to modify the environment.
- **Prototype:** `int putenv(char *string);`
- The string must be in the format "NAME=VALUE" and must persist.

Coding Example_6: Adding a new environment using putenv

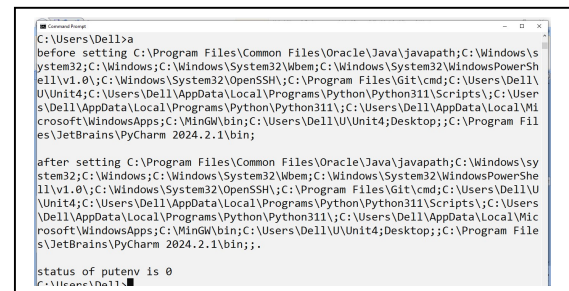
```
int main() {
    printf("%s\n", getenv("HOME"));
    putenv("HOME=pes");
    char *p = getenv("HOME");
    printf("home is %s", p);
    return 0;
}
```



```
C:\Users\De11>a
(null)
home is pes
```

Coding Example_7: Appending the new path to existing PATH

```
int main() {
    char *path = getenv("PATH");
    char mypath[10000];
    strcpy(mypath, path); strcat(mypath, ";");
    char path1[10000];
    strcpy(path1, "PATH="); strcat(path1, mypath);
    printf("before setting %s\n", getenv("PATH"));
    int i = putenv(path1);
    printf("\nafter setting %s\n", getenv("PATH"));
    printf("\nstatus of putenv is %d", i);
    return 0;
}
```



```
C:\Users\De11>a
before setting C:\Program Files\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files\Git\cmd;C:\Users\De11\U\Unit4;C:\Users\De11\AppData\Local\Programs\Python\Python311\Scripts\;C:\Users\De11\AppData\Local\Programs\Python\Python311\;C:\Users\De11\AppData\Local\Microsoft\WindowsApps;C:\WinGW\bin;C:\Users\De11\U\Unit4\Desktop;C:\Program Files\JetBrains\PyCharm 2024.2.1\bin;
after setting C:\Program Files\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files\Git\cmd;C:\Users\De11\U\Unit4;C:\Users\De11\AppData\Local\Programs\Python\Python311\Scripts\;C:\Users\De11\AppData\Local\Programs\Python\Python311\;C:\Users\De11\AppData\Local\Microsoft\WindowsApps;C:\WinGW\bin;C:\Users\De11\U\Unit4\Desktop;C:\Program Files\JetBrains\PyCharm 2024.2.1\bin;;
status of putenv is 0
```

Try this -> Display the list of all environment variables once after the new variable is added.

Modifying environment variables just got simpler !!